

Contents

1	Introduction	2
2	Teensy 4.0	3
3	TMC429	4
4	TMC2209	5
5	PCF8574	6
6	Buttons	6
7	Joystick	7
8	Selector switches	7
9	Limit switches	8
10	RGB Status LED	10
11	Mounting	10
12	Onboard voltage regulator	10
13	Miscellaneous information	10
14	Formulas	11
15	Serial commands	12
16	Useful links	13

1 Introduction

This system is a 3-axis stepper motor controller based on a Teensy 4.0 high-performance microcontroller, a TMC429 ramp generator and three TMC2209 stepper motor drivers.

The system can be controlled in a standalone mode using buttons or a joystick, but it can also be controlled via USB by sending specially formatted commands via a serial terminal software.

The board can be powered with up to 24 V (DC) power supply that can supply at least 3 A. The external power supply can be connected by a 2.5 x 5.5 mm center-positive DC barrel jack. The external power supply directly powers the three stepper motor drivers, and it also supplies power to a DC-DC converter that provides 3.3 V logical-level voltage for the rest of the components except the Teensy 4.0. The Teensy 4.0 microcontroller must be powered by an external power supply. Either connect it to the computer or use a mobile phone charger to power the microcontroller. The best practice to power up the system is to first connect the external power supply to the board, then provide power for the Teensy 4.0. This way, the microcontroller can properly connect to the external peripherals and modules, and it can initialize the stage.

All the GPIOs of the microcontroller are broken out, so the system is capable of accepting more modules or other external peripherals, for example a display.

The system can use limit switches on all three axes. The limit switches can be used both for homing and as a safety feature to stop the motor when a switch is triggered.

Disclaimer:

I hereby declare that this document and the accompanying software have been developed with sincere and benevolent intentions. Despite diligent efforts to ensure accuracy, it is acknowledged that errors or omissions may inadvertently arise. Consequently, I hereby absolve myself of any liability or responsibility for damages or adverse consequences resulting from the utilization of the provided resources.

The software is expressly designed for non-commercial applications, intended solely for recreational use and educational demonstrations. Its distribution is restricted to private individuals, namely hobbyists, and academic institutions, including universities and research institutes. Any unauthorized commercial use of the software is strictly prohibited and may result in legal action.

By accessing and using the software provided by **DM Devices**, you agree not to modify, resell, or commercially exploit the received source code and design files. These materials are solely for internal use and may not be distributed or used for any commercial purposes without explicit consent.

The users implicitly agree to abide by these terms and conditions, acknowledging the limitations and restrictions imposed herein.

May 16, 2024

2 Teensy 4.0

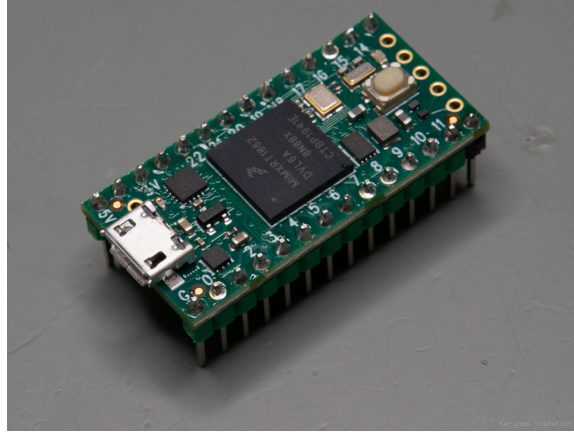


Figure 1: Teensy 4.0 microcontroller

The Teensy 4.0 is a fast and versatile microcontroller. It has sufficient number of GPIO pins, but most importantly, it has a lot of memory and a high CPU clock speed. For example, the TMC429 ramp generator board requires an external clock signal. This signal is a 32 MHz PWM signal provided by the Teensy 4.0. To make this 32 MHz as precise as possible, when the source code is compiled, the CPU Speed must be set to 528 MHz.

The board also has multiple PWM pins, analog pins, i2C and SPI ports which all come in handy in this project. The PWM pins are used for the RGB LED, the analog pins take care of the joystick and the i2C and SPI connections are used to communicate with the other modules. Furthermore, one of the additional UART ports are also used to communicate with the three TMC2209 stepper motor drivers. The Teensy 4.0 is powered via USB, either via a computer or by a power supply (e.g. a phone charger).

3 TMC429

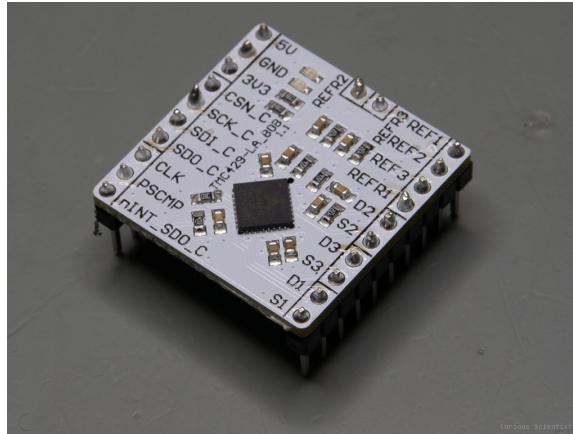


Figure 2: TMC429 ramp generator

This is a 3-axis ramp generator for stepper motors. It connects to the microcontroller (MCU) via SPI and then it can send the step and direction signals to three stepper motors simultaneously while also taking care of a total of six limit switches (2 per each motor). Its great property is that it totally unloads the MCU because the MCU only has to send a command to the TMC429, and then the TMC429 takes care of everything. It drives the motor according to a trapezoidal speed profile which is configured by the user. It automatically generates the pulses for the step pin and it keeps track of the number of steps being sent to each motor. So, it keeps track of the position of the motors. There's no need for constantly polling the step pin, or doing expensive mathematical operations to generate proper acceleration profiles. After the MCU sent the SPI message to the TMC429, there's no more load on it.

4 TMC2209

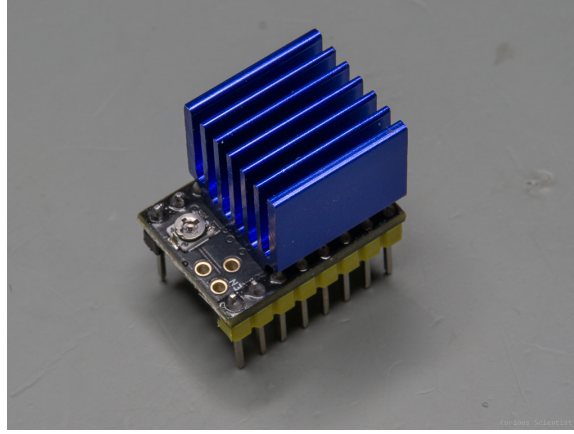


Figure 3: TMC2209 stepper motor driver with a heatsink

These drivers are one of the best stepper motor drivers today for projects like this. They are much better than the A4988 or the DRV8825 stepper motor drivers. They have a very smart power management which allows lower power consumption, more silent operation and less heating of the motors (and the drivers, too). They have an extremely good microstepping which makes them even more silent. They can communicate via UART, so in fact, they could be driven without step and direction signals, at least in simple scenarios.

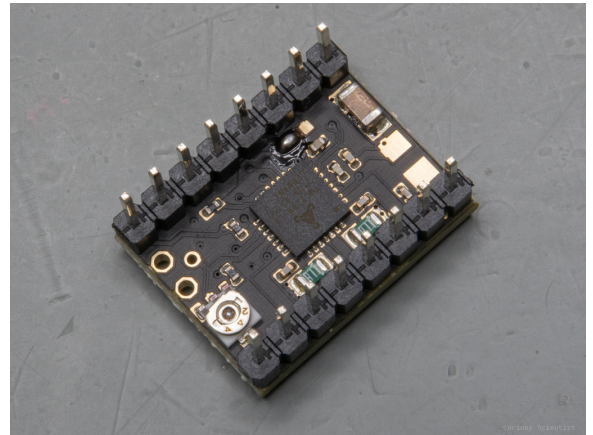
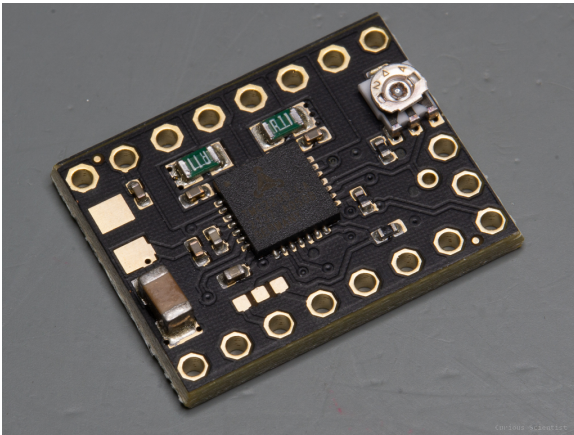


Figure 4: SilentStepStick TMC2209 module. Without solder bridge (left) and with solder bridge (right).

It is important to notice that the board is developed in a way that it is compatible with multiple versions of the TMC2209 stepper motor driver. In order to make it work with the original SilentStepStick module, a small adjustment on the module must be carried out. On the module, at one of the corners of the chip, there are three solder tabs (see Figure 4). By shorting the center tab to one of the other two tabs, the UART pin is either connected to pin 11 (PDN) or pin 12 (UART) on the module. It needs to be connected to pin 12, therefore the center solder tab must be shorted to the one on its right side. It is recommended to make this modification first before soldering in the other pins.

The polarity of each motor driver is the same. From left to right, each pair of pins belong to one coil of the stepper motor. According to the SilentStepStick driver, the pin layout is M1B, M1A, M2A, M2B, where M1B and M1A are one coil and M2B and M2A are another coil. Connect the stepper motor accordingly. If the motor rotates in the opposite way, swap the coils.

5 PCF8574

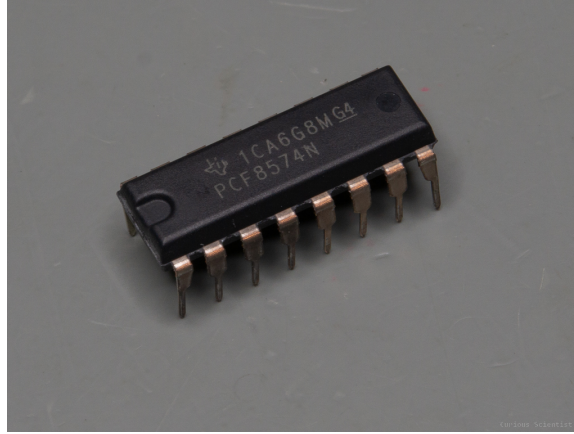


Figure 5: PCF8574 GPIO chip

This is a GPIO expander module. It is used in the circuit because I did not want to sacrifice six valuable GPIO pins from the Teensy 4.0, especially because those pins can be used for much more useful purposes than just simply reading a few switches. These are 6 pins which are the 2x3 pins for the axis selector and the mode selector switches. They are polled, and there's no need for high response speed (interrupt). The chip communicates with the MCU via i2C. Since the module has a total of 8 pins, the remaining 2 pins were also made available on the PCB. It can be useful if there are some other switches or buttons that need to be read. A suggestion would be a "speed multiplier button". If one of the remaining pins is active, it would be the "1x speed" option and if the other one is active, it would be the "5x speed" option. So, for example, when the user uses the joystick or the buttons to navigate the motors, it would be easy to quickly change speeds.

6 Buttons

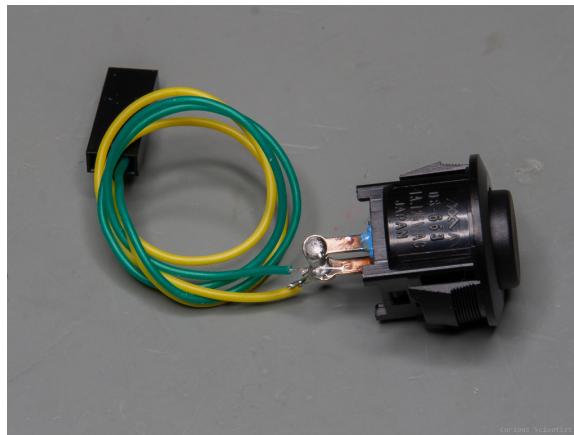


Figure 6: High-quality push button

The module has 3 pre-installed buttons: Abort, backward (BWD) and forward (FWD). These buttons are active when the module is used in button-controlled mode using the mode selector switch. The role of the buttons are self explanatory. The FWD button drives the selected axis in the positive direction and the BWD button runs in the negative direction. The abort button aborts the ongoing movements. If the abort button is pressed when none of the motors are moving, the system performs homing, one axis at a time.

7 Joystick

The module uses a 3-axis joystick to control the three stepper motors. The system responds to the joystick only when the joystick mode is active. When the module is powered on, the microcontroller performs an initial calibration on the 3 axes of the joystick to register their reference values. It is important to have the joystick in its default position before powering the device up!

The joystick has 5 kOhm resistance potentiometers on all three axes. All three axes are spring-loaded, so they automatically return to their initial position upon being released. The X and Y axes are controlled by tilting the joystick and the Z axis is controlled by twisting the cap of the joystick.

8 Selector switches

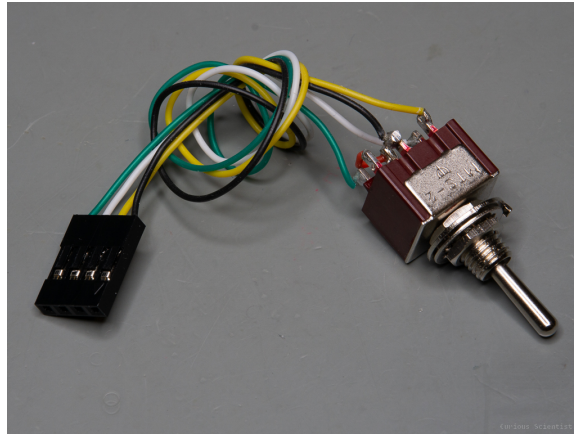


Figure 7: ON-ON-ON Toggle switch

The module has two, 3-position selector switches. One switch selects the control modes and the other selects the active axis when the module is used in button mode.

The three modes are the following:

- Joystick mode - the user can control the 3 axes with a 3-axis joystick
- Button mode - the user can select an axis and move it in the positive or negative direction with buttons
- USB mode - the user can send commands and parameters from the computer to control the 3 axes

Each switch has a notch on its threaded part. When the switch is at this notch, the first option is active (X-axis, joystick mode). When the switch is in the center position, the second option is active (Y-axis, button mode). And in the third position, the third option is active (Z-axis, USB mode), depending on which switch we look at.

9 Limit switches

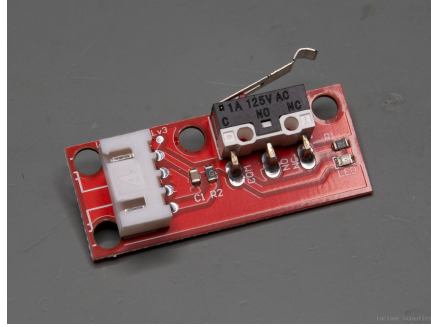


Figure 8: Typical limit switch module

The system has 6 limit switches, and each axis has 2 switches. Each switch acts as a stop switch. If they are hit during the movement of the motor, the motor stops and then it parks. Furthermore, each axis has a switch (in this application, it is set to be the left switch) that acts as the reference/home switch. Upon startup, each motor goes to its home position and parks using the left switch of the respective axis. After homing, each axis is considered to be at its home position at 0 mm.

It is important to pay attention that the limit switches should be connected to the board, otherwise the code on the microcontroller won't start properly. Alternatively, the user can remove the switch-related lines from the code, however, this is not encouraged. To simplify omitting the limit switches, the code was written in a way that the user can simply disable all the limit switch-related lines in the code that would prohibit the system from starting if there's a missing limit switch. Keep in mind that this disables features such as homing and parking after a limit switch is hit while moving. However, due to the nature of the TMC429 circuit, if a limit switch is pressed while moving on the corresponding axis, the motor will be blocked. If the property below is set to false, the code won't search for the limit switches upon booting. If the user needs the limit switches, the variable should be set to true.

```
1 bool limitSwitchesEnabled = false;
```

The limit switches must be high by default and when the limit switch is pressed, it should pull the signal pin to ground (low). To change this behaviour, please refer to the datasheet of the TMC429 ramp generator and check page 33. Switching between active high and active low modes is also indicated in the source code. It is important that ALL switches must behave in the same way, it is not allowed to use one switch with active high configuration and another with active low configuration.

If not this type (see, Figure 8)) of limit switch is used, the user has to make sure that the proper pull-up resistor is installed in the circuit of the limit switch. The unpressed limit switch should result in 3.3 V on the signal (S) pin of the limit switches and in addition to this, the signal pin should be connected to the ground via a 4.7 kOhm resistor.

When a limit switch is hit while moving the motor, the motor immediately stops and parks nearby the limit switch. The status LED turns red. After a limit switch was hit, the system will not respond to any further instructions on the axis that tripped its limit switch until it is unlocked. To unlock the axis, move the mode selector or axis selector switch to a different position. When the light turns to blue, the system is unlocked.

It is good to know that during homing, the stepper motors approach their home position slower than they move otherwise. After the motors are parked, the original speed settings are restored. It is important to keep in mind that these speed values depend on the mechanism and the arrangement of the motor. The user must adjust these values to achieve the desired behavior!

10 RGB Status LED

The status of the controller is indicated with an RGB LED. Each colour of the LED is driven by a PWM signal. Depending on the magnitude and the proportion of the three PWM signals, the LED will show a colour. The colours have the following meanings:

- Blue - Standby. The motors are standstill and the module is waiting.
- Blinking blue - GPIO status change. Axis or mode selector switch changed.
- Green - In progress. One of the motors are moving.
- Red - Limit switch is active. One of the six limit switches were hit.
- Blinking red - Abort. Abort button was pressed.
- Orange - Communication error. The TMC429 module is not responding

11 Mounting

The board has a padded M4 mounting hole at each of its corners. To preserve the longevity of the board, plastic washers must be used on both sides when mounting the board.

12 Onboard voltage regulator

The onboard voltage regulator is an off-the-shelf MP2315 chip-based adjustable voltage regulator. The voltage regulator is fed from the input voltage line, the same line that feeds the three TMC2209 stepper motor drivers. It takes up to 24 V.

Its output voltage is fixed at 3.3 V and it provides logic power to each components on the board, except for the Teensy 4.0. Its maximum output current is 3 A. The circuit is (intentionally) designed this way that it does not power the Teensy 4.0 microcontroller. Since the board can be controlled from a computer via the serial terminal, it is assumed that it is constantly connected to one. To avoid any potential damage to the circuit by providing power to it from multiple sources, the 3.3 V power line of the board is not connected to the Teensy 4.0 microcontroller. If the board is used in a standalone mode, a simple mobile phone charger is sufficient to power the Teensy 4.0.

13 Miscellaneous information

When powering the device on for the first time, make sure that all three axes have enough range of motion before hitting anything while moving. This is advised because different motors and systems might respond different to the same speed settings (especially, if the attached mechanisms are different). Furthermore, based on the wiring, the axes might be inverted.

Make sure that the motor polarity is physically matched with the stage and its motors in order to achieve correct behaviour. When the motor moves in the negative direction, the stage should move towards the respective motor and the left hand switch (switch placed at the motor, a.k.a. home switch). This is particularly important because of the homing procedure. When the stage performs homing, the motor moves in the negative direction, towards the motor that moves the active axis.

Due to the complexity of the homing procedure, the homing (as of now) cannot be cancelled/aborted. Make sure that the system can perform homing when it is instructed to do so. To abort homing, remove the power supply of the MCU. This also resets the whole system.

14 Formulas

For clarity, the formulas used in positioning are discussed below.

It is assumed that the stepper motors drive a linear mechanism such as a lead screw. To be able to tell the motor to move "x amount of millimeters", the following parameters must be considered:

- Number of (micro)steps per revolution
- Pitch of the lead screw's thread
- Target position in millimeters

When the user enters the target position in millimeters, the software converts the distance into the number of steps that is necessary to go to the requested target position. A typical stepper motor has 200 steps/rev or 1.8°/step resolution. If the stepper motor driver is driven at the "full step" setting, then the motor will do 200 steps to complete a full turn. If the motor was attached to a lead screw with 1 mm pitch, this would mean that it took 200 steps to generate 1 mm linear displacement. More precisely, the nut attached to the linear screw would travel 1 mm. So, with simple math we can see that 1 step is 5 μ m or 0.005 mm linear displacement. Once the step-to-millimeters factor is know, any target position given in millimeters can be reached.

$$D = \mu \cdot n \cdot p, \tag{1}$$

where D is the travel distance of the linear rail using a lead screw, μ is the microstepping setting, n is the number of steps per turn resolution of the stepper motor, p is the pitch of the lead screw in millimeters.

It is worth noting that the stepper motor drivers are typically used with some microstepping setting. The current code is configured to us a factor of 16 microstepping which means that a typical 200 steps/rev stepper motor will take $200 \cdot 16 = 3200$ steps to complete a full turn. Or, one step will result in 312.5 nm linear displacement. Keep in mind that the system might suffer from backlash or other mechanical imperfections that might impair its ability to move at such high resolution!

Keep in mind that the speed and acceleration values must be scaled with the selected microstepping value. If a motor's speed is set to 800 steps/s at the factor of 8 microstepping, the speed of the motor will be half of its speed when the microstepping factor is set to 16 and the speed is left unchanged. This is due to the fact that the motor now has twice as much steps to complete, but the time between two steps is still the same as previously when the microstepping was at 8.

15 Serial commands

- Acceleration
 - Command: **a2000** sets the acceleration to 2000 steps/s² on all three axes.
- Velocity
 - Command: **v200** sets the velocity to 200 steps/s on all three axes.
- X position
 - Command: **x4.2** moves the X-axis to 4.2 mm absolute position¹.
- Y position
 - Command: **y2.7** moves the Y-axis to 2.7 mm absolute position¹.
- Z position
 - Command: **z0.9** moves the Z-axis to 0.9 mm absolute position¹.
- X-Y position
 - Command: **p3.24,4.56** moves the X-axis to 3.24 mm absolute position and the Y-axis to 4.56 mm absolute position¹.
- Homing
 - Command: **hi** moves the i-axis to the home position and parks the motor.
X-axis: **h0**, Y-axis: **h1**, Z-axis: **h2**.
- Communication test
 - Command: **T** checks the connection between the MCU and the TMC249 module.
- Reset
 - Command: **R** resets the MCU and reinitializes the system.

¹Please notice that the positioning depends on the actual mechanism. Currently, it is assumed that the stepper motors are connected to a lead screw with 1 mm pitch.

16 Useful links

[TMC429 Arduino library](#)
[TMC2209 Arduino library](#)
[TMC429 Datasheet](#)
[TMC2209 Datasheet](#)
[Teensy 4.0 Website](#)